



Fakultät für Maschinenbau und Mechatronik
Studiengang Robotik und KI i.d. Produktion

Projektbericht

Autonome Fahrzeugsteuerung für Carrera Hybrid mittels Deep Reinforcement Learning und Privilegierter Expertendestillation

Konzeption, 2D-Prototyping und empirische Evaluation einer sample-effizienten Teacher-Student-Architektur zur Vorbereitung des Sim-to-Real-Transfers

Vorgelegt von: Felix Faaß, Magnus Neumann

Matrikelnummer: [103121, 103560]

Betreuer (HKA): Prof. Dr.-Ing. habil. Björn Hein

Karlsruhe, den 31. August 2026

Eigenständigkeitserklärung

Wir versichern hiermit wahrheitsgemäß, dass wir die vorliegende Projektarbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder unveröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht.

Die Arbeit hat in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen und wurde nicht anderweitig veröffentlicht.

Karlsruhe, den [Datum der Abgabe]

(Felix Faaß)

(Magnus Neumann)

Kurzfassung

Das Training von End-to-End Reinforcement Learning (RL) Agenten zur visuellen Fahrzeugsteuerung am physikalischen Limit ist in fotorealistischen 3D-Simulatoren mit extrem hohem Rechenaufwand und ausgeprägter Sample-Ineffizienz verbunden. Die vorliegende Projektarbeit adressiert diese Herausforderung durch den Aufbau einer leichtgewichtigen, physikalisch kalibrierten 2D-Simulationsumgebung in Python zur iterativen Methodenentwicklung und Evaluation von RL-Architekturen für *Carrera Hybrid* Rennfahrzeuge (Maßstab ca. 1:43).

Als zentraler methodischer Ansatz wird eine **Cross-Modal Privileged Teacher-Student Distillation Pipeline** erforscht: Ein leichtgewichtiger Lehrer-Agent (Oracle) wird binnen weniger Minuten auf niedrigdimensionalen Raycast-Sensoren trainiert. Durch gezielte Rausch-Injektion (*Action Noise Injection*) während der Demonstrationsfahrten generiert der Lehrer einen Datensatz reich an Rettungsmanövern (*Recovery Behaviors*), wodurch das fundamentale Problem des *Covariate Shift* beim anschließenden *Behavioral Cloning* des visuellen Convolutional Neural Networks (CNN) gelöst wird. Das vortrainierte visuelle Modell wird anschließend mittels *Soft Actor-Critic* (SAC) auf Rundenzeiten optimiert.

Die empirische Untersuchung deckt zentrale physikalische und wahrnehmungstheoretische Phänomene auf: Insbesondere wird das **Subpixel-Wahrnehmungsproblem** bei realistischen Fahrzeuggeschwindigkeiten mathematisch hergeleitet und experimentell validiert. Ein isolierter Kontrollversuch belegt, dass SAC unter gemessener Fahrzeugphysik die Rennstrecke mit Lidar-Sensorik in 54 Minuten meistert (über 11 fehlerfreie Runden), während reine Bildagenten bei Standard-Framestapeln an der Bewegungswahrnehmungsschwelle scheitern. Zur Lösung werden temporale Striding-Verfahren und optimierte Replay-Buffer-Architekturen vorgestellt. Die Arbeit liefert damit die validierte methodische Grundlage für den anschließenden Transfer in NVIDIA Isaac Sim und das reale Fahrzeug-Deployment.

Abstract

Training end-to-end vision-based reinforcement learning (RL) agents for autonomous racing at the physical friction limit typically requires prohibitive compute budgets and suffers from severe sample inefficiency in complex 3D simulators. This project addresses these challenges by developing a lightweight, physics-calibrated 2D simulation environment in Python for rapid prototyping, architecture exploration, and benchmark evaluation targeting *Carrera Hybrid* miniature race cars (1:43 scale).

The core methodological contribution is a **Cross-Modal Privileged Teacher-Student Distillation Pipeline**: A lightweight teacher agent (oracle) is trained within minutes using low-dimensional raycast sensors. By introducing deliberate *Action Noise Injection* during demonstration rollouts, the teacher generates a diverse dataset containing essential recovery maneuvers, effectively mitigating the *Covariate Shift* during subsequent *Behavioral Cloning* of the visual Convolutional Neural Network (CNN). The pretrained vision backbone is then fine-tuned using *Soft Actor-Critic* (SAC) for lap-time optimization.

Empirical evaluations reveal critical physical and perceptual bottlenecks: Specifically, the **subpixel perception barrier** at realistic miniature vehicle velocities is mathematically derived and experimentally verified. An isolated control experiment proves that SAC seamlessly solves the racing task under measured vehicle dynamics with lidar inputs in 54 minutes (>11 consecutive laps), isolating visual motion resolution as the sole performance bottleneck. To resolve this, temporal frame striding and memory-efficient replay buffer architectures are established. This work provides the validated methodology and architectural blueprints for subsequent deployment in NVIDIA Isaac Sim and real-world sim-to-real transfer.

Inhaltsverzeichnis

- 1. Einleitung
 - 1.1 Motivation und Anwendungsbereich
 - 1.2 Projektstruktur und forschungsmethodische Einordnung
 - 1.3 Zielsetzung und Anforderungskatalog
- 2. Theoretische Grundlagen und Stand der Technik
 - 2.1 Reinforcement Learning Grundlagen (MDP vs. POMDP)
 - 2.2 Zeitliche Dynamik und Frame Stacking
 - 2.3 Imitation Learning, Behavioral Cloning und Covariate Shift
 - 2.4 Privilegiertes Lernen: Teacher-Student Distillation
 - 2.5 Algorithmen im kontinuierlichen Kontrollraum (PPO vs. SAC)
 - 2.6 State of the Art im autonomen Rennsport
- 3. Methodik und Aufbau der 2D-Simulationsumgebung
 - 3.1 Streckengeometrie und Computer Vision Pipeline
 - 3.2 Physikalische Modellierung: Von `force_drag_v1` zu `measured_v2`
 - 3.3 Beobachtungsräume und Virtuelle Kamera
 - 3.4 Aktionsraum und Tiefpass-Lenkodynamik
 - 3.5 Reward Engineering und Belohnungsgestaltung
- 4. Der 4-Phasen-Trainingsansatz (Teacher-Student Pipeline)
 - 4.1 Phase 1: Bootstrapping des Lidar-Experten
 - 4.2 Phase 2: Datensammlung mit Action Noise Injection
 - 4.3 Phase 3: Visuelles Pre-Training (Behavioral Cloning)
 - 4.4 Phase 4: End-to-End Fine-Tuning mit Soft Actor-Critic (SAC)
- 5. Experimentelle Ergebnisse und Chronologie der Trainingsläufe
 - 5.1 Chronologie der Experimente und Paradigmenwechsel
 - 5.2 Quantitativer Leistungsvergleich
 - 5.3 Der Kontrollversuch: Lidar-SAC unter gemessener Physik
- 6. Tiefgehende RL-Erkenntnisse und Systemanalysen
 - 6.1 Das Subpixel-Wahrnehmungsproblem
 - 6.2 Lösungsstrategien: Temporale Spreizung vs. Bildauflösung
 - 6.3 Replay-Buffer-Architektur und Speicheroptimierung
 - 6.4 Korrekte Behandlung von Zeitschritt-Timeouts
 - 6.5 TensorBoard-Diagnostik: Critic-Divergenz vs. Ausreißer
 - 6.6 Messfallen und Rendering-Diskrepanzen
- 7. Übertragung in die 3D-Simulation (NVIDIA Isaac Sim)
 - 7.1 Aufbau der 3D-Simulationsumgebung in Isaac Lab [Platzhalter]
 - 7.2 Asymmetric Actor-Critic und Domain Randomization
 - 7.3 Evaluierung und Sim-to-Real Ausblick [Platzhalter]
- 8. Fazit, Learnings und Ausblick
 - 8.1 Zusammenfassung der Ergebnisse
 - 8.2 Zentrale Erkenntnisse (Key Learnings)
 - 8.3 Ausblick auf das Folgeprojekt
- Quellenverzeichnis
- Abbildungs- und Tabellenverzeichnis

- Anhang

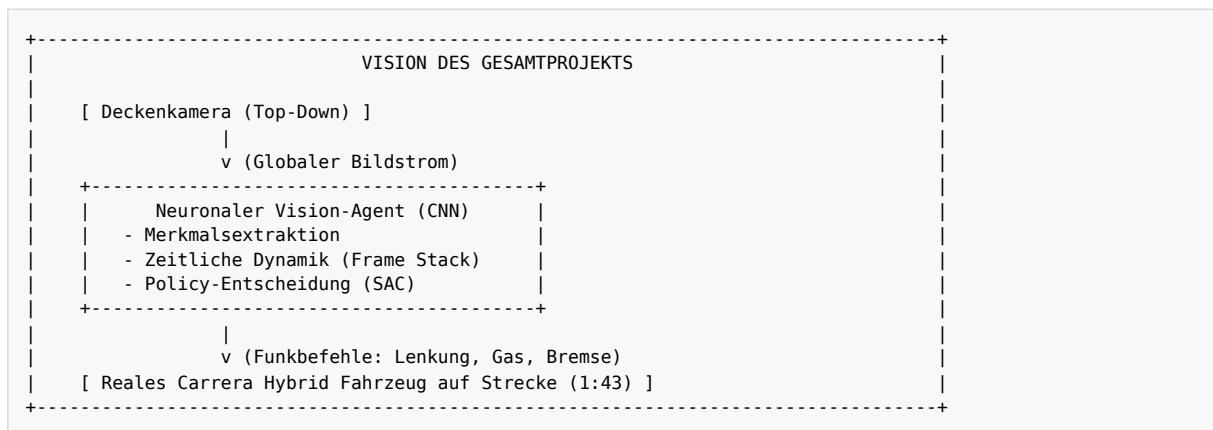
1. Einleitung

1.1 Motivation und Anwendungsbereich

Das autonome Fahren im physikalischen Grenzbereich stellt eine der anspruchsvollsten Disziplinen der modernen Robotik und Künstlichen Intelligenz dar. Während klassische regelungstechnische Ansätze wie modellprädiktive Regelungen (Model Predictive Control, MPC) bei präzise bekannten Strecken- und Reibungsparametern hervorragende Ergebnisse erzielen, stoßen sie bei hochdynamischen Effekten (Reifenschlupf, Lastwechselreaktionen, Drift-Übergänge) und unvollständiger Sensorik an fundamentale Grenzen.

Deep Reinforcement Learning (DRL) bietet hierbei das theoretische Potenzial, nichtlineare Fahrdynamiken rein datengetrieben direkt aus Sensordaten zu erlernen. Als ideales Experimentierfeld dient in dieser Arbeit das System **Carrera Hybrid**: Miniatur-Rennfahrzeuge im Maßstab 1:43, die sich frei (ohne mechanische Führungsschiene) auf einer Fahrbahn bewegen und über kontinuierliche Lenk- und Antriebsaktoren gesteuert werden.

In der angestrebten Gesamtvision überwacht eine stationäre Deckenkamera (Top-Down Bird's Eye View) die gesamte Rennstrecke. Ein neuronaler Vision-Agent verarbeitet diesen globalen Bildstrom, erkennt die Position und Ausrichtung des Fahrzeugs relativ zu den Streckengrenzen und steuert das Fahrzeug mit maximaler Geschwindigkeit entlang der Ideallinie.



1.2 Projektstruktur und forschungsmethodische Einordnung

Die vorliegende Forschungs- und Entwicklungsarbeit behandelt die **erste Phase** dieses Gesamtvorhabens. Das direkte Training eines visuellen End-to-End-Agenten in komplexen 3D-Simulatoren (wie NVIDIA Isaac Sim / Isaac Lab) ist rechenintensiv, leidet unter langen Feedbackzyklen und scheitert in der Praxis häufig an unzureichender Sample-Effizienz.

Daher verfolgt dieses Projekt einen systematischen zweistufigen Forschungsansatz: 1. **Phase 1 (Gegenstand dieser Arbeit): 2D-Python-Prototyping & Methodenevaluation**

Aufbau einer leichtgewichtigen, hochperformanten 2D-Simulationsumgebung (**Carrera2DEnv**), um fundamentale Fragen zu Beobachtungsräumen, Netzwerkarchitekturen, Lernalgorithmen (PPO vs. SAC), Belohnungsfunktionen und Datenvorverarbeitungsmethoden isoliert und zeiteffizient zu untersuchen. 2. **Phase 2 (Transfer & Sim-to-Real):**

Übertragung der im 2D-Prototyping validierten Best Practices in die fotorealistische 3D-Simulation (NVIDIA Isaac Sim) und anschließende Erforschung des Sim-to-Real-Gaps auf der realen Carrera-Hybrid-Hardware.

1.3 Zielsetzung und Anforderungskatalog

Die konkreten Zielsetzungen für die prototypische Entwicklungsphase lauten: * **Entwicklung einer physikalisch kalibrierten 2D-Umgebung:** Realitätsgetreue Abbildung der Streckenmaße (25 cm Fahrbahnbreite, 21 mm Randstreifen) und gemessenen Fahrzeugbeschleunigungen. * **Architekturvergleich verschiedener Inputmodalitäten:** Systematischer Vergleich von niedrigdimensionalen Raycasts (Lidar), Einzelbildern, fahrzeugfesten Bildausschnitten (Ego-Crops) und globalen gestapelten Kamerabildern. * **Entwicklung einer stabilen Teacher-Student-Distillationspipeline:** Effizientes Vortraining visueller Netze durch automatisierte Expertendemonstrationen. * **Rundenzeit- und Stabilitätsoptimierung:** Erreichen fehlerfreier Mehrfachrunden am physikalischen Traktionslimit.

2. Theoretische Grundlagen und Stand der Technik

2.1 Reinforcement Learning Grundlagen (MDP vs. POMDP)

Ein klassisches Reinforcement-Learning-Problem wird als **Markov-Entscheidungsprozess** (MDP) formuliert, definiert durch das 5-Tupel $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$: * $\mathcal{S}$: Zustandsraum (State Space) * $\mathcal{A}$: Aktionsraum (Action Space) * $\mathcal{P}(s' | s, a)$: Übergangswahrscheinlichkeitsfunktion (Transition Dynamics) * $\mathcal{R}(s, a, s')$: Belohnungsfunktion (Reward Function) * $\gamma \in [0, 1]$: Diskontierungsfaktor für zukünftige Belohnungen (Discount Factor)

Ziel des Agenten ist die Maximierung des diskontierten kumulativen Ertrags (Return) $R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$.

Liegen dem Agenten nicht alle Zustandsvariablen (z. B. translatorische und rotatorische Geschwindigkeiten, Reibwerte) direkt vor, sondern lediglich unvollständige Beobachtungen $o \in \Omega$ aus einer Beobachtungsfunktion $\mathcal{O}(s)$, liegt ein **Partiell Beobachtbarer MDP** (POMDP) vor.

2.2 Zeitliche Dynamik und Frame Stacking

Ein einzelnes, statisches Graustufenbild $o_t \in \mathbb{R}^{H \times W}$ ist fundamental nicht-markovianisch: Aus einem einzelnen Foto lässt sich nicht ableiten, ob das Fahrzeug mit **1.5 m/s** vorwärtsfährt, rückwärts rollt oder stillsteht.

Durch das **Frame Stacking** von k aufeinanderfolgenden Bildern:

$$s_t = [o_t, o_{t-1}, \dots, o_{t-k+1}] \in \mathbb{R}^{k \times H \times W}$$

wird die zeitliche Pixelverschiebung $\Delta x / \Delta t$ über die Kanaldimension für Faltungsnetzwerke (CNNs) direkt lesbar. Faltungsfilter in den ersten Netzwerkschichten erlernen dadurch die Berechnung räumlich-zeitlicher Gradienten, was den POMDP näherungsweise in einen MDP überführt (mnih2015human).

2.3 Imitation Learning, Behavioral Cloning und Covariate Shift

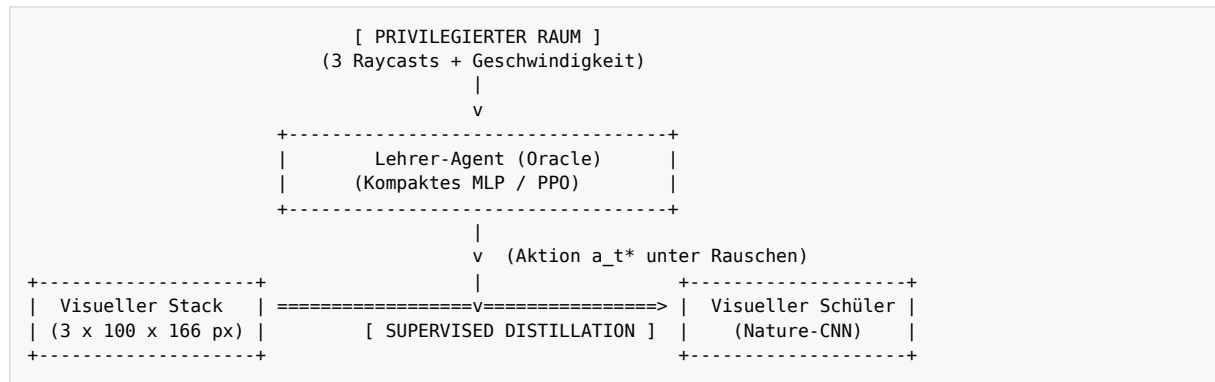
Beim **Behavioral Cloning** (BC) lernt ein parametrisiertes Modell π_θ mittels überwachten Lernens (Supervised Learning) die direkte Abbildung von Zuständen auf Expertenaktionen $\mathcal{L}_{\text{BC}}(\theta) = \frac{1}{N} \sum \|\pi_\theta(s_i) - a_i\|^2$

Das fundamentale Versagen von naivem BC: Ein optimaler Experte erzeugt ausschließlich Trajektorien auf der Ideallinie $\mathcal{S}_{\text{ideal}}$

. Durch unvermeidliche Approximationsfehler driftet der Schüler nach wenigen Zeitschritten in unbekannte Randzustände $s \notin \mathcal{S}_{\text{ideal}}$ ab (Out-of-Distribution). Da im Trainingsdatensatz keine Korrekturmanöver für s existieren, was zum unmittelbaren Streckenabflug führt, enthalten sich die Fehler quadratisch über den Zeithorizont $\mathcal{O}(T^2)$ nach Ross & Bagnell (ross2011reduction)

2.4 Privilegiertes Lernen: Teacher-Student Distillation

Das Paradigma des *Privileged Learning* bzw. *Learning by Cheating* (Chen et al. chen2019learning) entkoppelt das Steuerungsproblem vom Wahrnehmungsproblem: 1. **Schritt 1 (Lehrer / Oracle):** Training einer Policy π_{teacher} auf einfachen, perfekten Sensorwerten (z. B. Raycast-Distanzen). Da die Dimensionalität gering ist, konvergiert dieser Schritt in kürzester Zeit. 2. **Schritt 2 (Schüler / Student):** Training eines visuellen Modells π_{student} , das auf Rohpixeln operiert und lernt, die Aktionen des Lehrers zu replizieren.



2.5 Algorithmen im kontinuierlichen Kontrollraum (PPO vs. SAC)

- **Proximal Policy Optimization (PPO)** [\cite{schulman2017proximal}](#): On-Policy Algorithmus, der Stabilität durch eine geclippte Wahrscheinlichkeitsquotienten-Zielfunktion erzielt. Verwirft gesammelte Transitions nach jedem Gradientenupdate, was bei bildbasierten Eingaben zu hoher Sample-Ineffizienz führt.
- **Soft Actor-Critic (SAC)** [\cite{haarnoja2018soft}](#): Off-Policy Actor-Critic Algorithmus, der neben dem erwarteten Return die Entropie der Policy maximiert: $J(\pi) = \mathbb{E}\{\tau \sim \pi\} \left[\sum (\pi(\cdot \mid s_t)) \right]$. SAC nutzt einen Replay-Buffer zur wiederholten Auswertung von Erfahrungen und verhindert durch $\sum \gamma^t \left(r(s_t, a_t) + \alpha \mathcal{H} \text{Double-Q-Learning} \right)$ (zwei parallele Critic-Netzwerke $Q_{\theta_1}, Q_{\theta_2}$) systematische Q-Wert-Überschätzungen.

2.6 State of the Art im autonomen Rennsport

Die Konzeption dieser Arbeit stützt sich auf zentrale Erkenntnisse führender Publikationen: * **DIRL** (Cai et al. [\cite{cai2021vision}](#)): Beweist die Überlegenheit von hybridem Imitation- und Reinforcement-Learning für reale 1:20 RC-Fahrzeuge. * **Gran Turismo Sophy** (Wurman et al., Nature 2022 [\cite{wurman2022outracing}](#)): Erreicht übermenschliche Rundenzeiten mittels SAC, asymmetrischen Critic-Architekturen und optimierten Streckenfortschritts-Belohnungen. * **VISTA** (Amini et al. [\cite{amini2020learning}](#)): Zeigt, dass das gezielte Erleben und Trainieren von OOD-Rettungsmanövern für einen robusten Sim-to-Real-Transfer essenziell ist.

3. Methodik und Aufbau der 2D-Simulationsumgebung

3.1 Streckengeometrie und Computer Vision Pipeline

Die Simulationsumgebung basiert auf der realen Geometrie einer Carrera-Hybrid-Rennstrecke: * **Fahrbahnbreite: 250 mm** * **Randstreifen: 21 mm** (Farbmarkierungen: Rot für Außenrand, Blau für Innenrand) * **Fahrzeugmaße:** Länge **98 mm**, Breite **39 mm**, Achsabstände: Heck bis Hinterachse **23 mm**, Heck bis Vorderachse **71 mm**.

Das Modul tracklimits_cv.py extrahiert über Farbmasken im HSV-Farbraum die Konturen der Begrenzungen. Aus den Nearest-Neighbor-Mittelpunkten zwischen Innen- und Außenrand wird über B-Spline-Interpolation eine stetig differenzierbare **Mittellinie** generiert, die als Referenz für Fahrtrichtung und Streckenfortschritt dient.

3.2 Physikalische Modellierung: Von force_drag_v1 zu measured_v2

Im Projektverlauf wurden zwei grundlegend verschiedene Physikmodelle untersucht:

Parameter	force_drag_v1 (Frühes Modell)	measured_v2 (Gemessenes Modell)	Reale Auswirkung
Max. Beschleunigung	12.5 m/s ²	0.6 – 1.2 m/s ²	Altes Modell war ca. 20-fach übermotorisiert .
Max. Bremsverzögerung	14.4 m/s ²	1.5 m/s ²	Altes Modell erlaubte unrealistisch abrupte Stopps.
Max. Endgeschwindigkeit	5.0 – 12.0 m/s	1.6 – 1.9 m/s	Anpassung an reale Carrera-Hybrid-Höchstgeschwindigkeit.
Rückwärtsfahren	Erlaubt ($v < 0$)	Gesperrt ($v \geq 0$)	Realitätsabgleich mit Carrera-Antriebsstrang.
Lenkänderungsrate $\Delta\delta$	1.0 /Step (Sprunghaft)	0.5 /Step (Tiefpass)	Modellierung der Servo-Stellzeit.

3.3 Beobachtungsräume und Virtuelle Kamera

Die Umgebung (carrera_2d_env.py) stellt modulare Beobachtungsräume bereit: 1. **Raycasts / Lidar (Privilegiert):** Vektor aus 3 Distanzstrahlen (Schnittpunkte mit Streckenbegrenzungen links, mitte, rechts) sowie der Eigengeschwindigkeit: $o_t = [v/v_{\max}, d_l, d_m, d_r] \in \mathbb{R}^4$. 2. **Globale Kamera (Bird's Eye View):** Skaliertes Graustufenbild der gesamten Rennstrecke (166×100 Pixel, quantisiert auf 4 Helligkeitsstufen zur Rauschunterdrückung). 3. **Gestapelte globale Ansicht (Standard):** Stack aus $n_{\text{stack}} = 3$ Bildern $\rightarrow \mathbb{R}^{3 \times 100 \times 166}$.

3.4 Aktionsraum und Tiefpass-Lenkndynamik

Kontinuierlicher Aktionsraum $\mathcal{A} \in [-1, 1]^2$: * $a[0] \in [-1, 1]$: Lenkwinkel (begrenzt durch $\Delta\delta \leq 0.5$ pro Simulationsschritt $\Delta t = 0.0167$ s). * $a[1] \in [-1, 1]$: Antrieb (> 0 : Motorleistung, < 0 : Bremse).

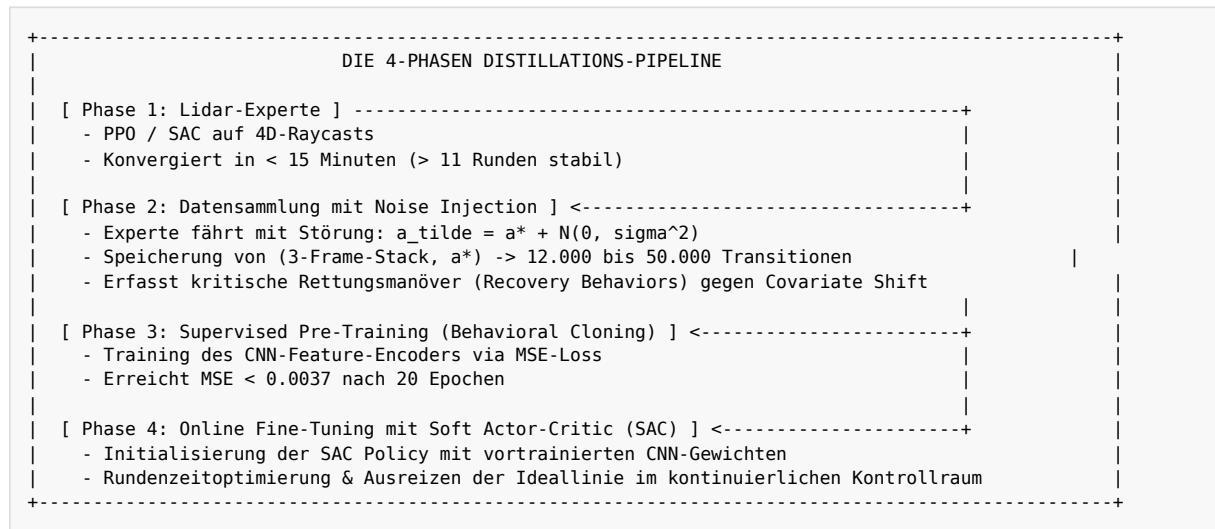
3.5 Reward Engineering und Belohnungsgestaltung

Die finale Belohnungsfunktion v3_rundenzeit (reward_func.py) balanciert Fortschritt und Sicherheit:

$$r_t = w_{\text{prog}} \cdot \Delta s_{\text{mittellinie}} + w_v \cdot \frac{v}{v_{\max}} - w_{\text{slow}} \cdot \max\left(0, 1 - \frac{v}{v_{\text{threshold}}}\right) - r_{\text{crash}} + r_{\text{lap_bonus}}$$

Wichtige Fehlerkorrekturen gegenüber früheren Versionen: * **Entfernung der Sperrklinke (Ratchet-Bug):** Frühere Rewards bestraften langsamere Runden relativ zu einer Zufalls-Bestzeit. * **Beseitigung von Oszillationsstrafen ($w_{\text{integral}} = 0.0$):** Die vormalige Bestrafung kontinuierlicher Lenkwinkel bestrafte saubere Kurvenfahrten mit **−625** Punkten pro Episode.

4. Der 4-Phasen-Trainingsansatz (Teacher-Student Pipeline)



4.1 Phase 1: Bootstrapping des Lidar-Experten

Aufgrund des kompakten Zustandsraums ($\mathbb{R}^4$) lernt ein zweischichtiges MLP ([64, 64]) mittels PPO oder SAC innerhalb von **50.000** Schritten (< 15 Minuten Rechenzeit) eine perfekte Fahrkompetenz mit Durchschnittsgeschwindigkeiten von **1.22 m/s**.

4.2 Phase 2: Datensammlung mit Action Noise Injection

Um den Schüler auf Abweichungen von der Ideallinie vorzubereiten, wird der Lidar-Experte stochastisch gestört: $\tilde{a}_t = \text{clip}(a_t + \xi_t, -1, 1)$, $\xi_t \sim \mathcal{N}(0, \sigma^2)$. Das Auto bricht aus, und der Lidar-Experte steuert im Folgeschritt mit maximaler Gegenlenkung gegen. Gespeichert wird das Paar (s_t, a_t) , bestehend aus dem visuellen 3-Frame-Stack s_t und der unverrauschten Korrekturaktion a_t^* .

4.3 Phase 3: Visuelles Pre-Training (Behavioral Cloning)

Ein Faltungsnetzwerk (Nature-CNN oder modifiziertes 3-Layer-CNN mit Linear-Kopf) wird per Gradientenabstieg optimiert: $\mathcal{L}(\theta) = \frac{1}{|D|} \sum_{(s, a) \in D} \|f_\theta(s) - a\|^2$. Nach 20 Epochen erreicht das Netzwerk einen Validierungs-MSE von **< 0.0037**, wodurch die visuellen Filter für Streckenränder und Fahrzeuglage bereits vor dem ersten RL-Schritt vollständig ausgebildet sind.

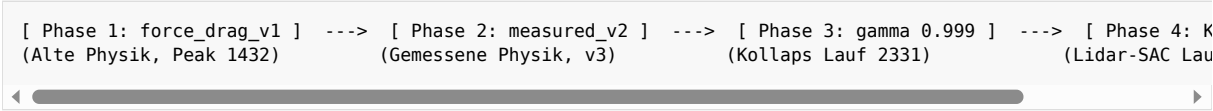
4.4 Phase 4: End-to-End Fine-Tuning mit Soft Actor-Critic (SAC)

Das vortrainierte Netzwerk wird als Actor-Backbone in SAC überführt. Der Off-Policy Replay-Buffer sammelt reale Fahrerfahrungen, während die Entropiemaximierung verhindert, dass der Agent in lokalen Minima (z. B. vorsichtiges Dauerschleichen) verharrt.

5. Experimentelle Ergebnisse und Chronologie der Trainingsläufe

5.1 Chronologie der Experimente und Paradigmenwechsel

Die Chronologie der Trainingsläufe dokumentiert die methodische Evolution des Projekts:



5.2 Quantitativer Leistungsvergleich

Alle Modelle wurden unter identischen Evaluationsbedingungen im aktuellen Benchmark-Runner (src/eval/vergleich.py) vermessen:

Modell / Run	Algorithmus	Beobachtungsraum	Physikmodell	Total Steps	Reward (Eval)	EpLen (Frames)	Runden	Ø-Geschw.	Status / Befund
Lidar-Experte	PPO	Raycasts (4D)	measured_v2	500 k	6401.7	2000 (Limit)	11	1.22 m/s	Souveräne Referenz
Lidar SAC (1748)	SAC	Raycasts (4D)	measured_v2	500 k	6116.4	2000 (Limit)	11	1.19 m/s	Kontrollversuch bestanden
Vision SAC 1432	SAC	3-Stack (166 × 100)	force_drag_v1	2.5 M	3828.2	2000	3	1.27 m/s	Peak alte Physik; divergiert später
Vision SAC 2210	SAC	3-Stack (166 × 100)	measured_v2	5.0 M	−30.3	256	0	0.81 m/s	Scheitert an Subpixel-Schwelle
Vision SAC 2147	SAC	3-Stack (166 × 100)	measured_v2	2.5 M	−266.5	174	0	0.62 m/s	Früher Wandkontakt
Vision SAC 2331	SAC	3-Stack (166 × 100)	measured_v2	1.5 M	−349.8	33	0	0.12 m/s	Kollaps durch $\gamma = 0.999$
Vision PPO 2328	PPO	3-Stack (166 × 100)	force_drag_v1	90 M	−3.9	76	0	0.45 m/s	Extreme Sample-Ineffizienz

5.3 Der Kontrollversuch: Lidar-SAC unter gemessener Physik

Um zu überprüfen, ob das Scheitern der visuellen Agenten unter measured_v2 an der SAC-Stabilität, der trägeren Physik oder dem Reward liegt, wurde in Lauf 1748 die visuelle Beobachtung gegen Lidar-Werte getauscht.

Ergebnis: Der Agent meisterte die Strecke in **54 Minuten** vollständig (**6116.4** Punkte, **11** Runden). Damit wurde zweifelsfrei bewiesen: **Physik, Reward und Algorithmus funktionieren perfekt – die Ursache liegt rein in der visuellen Wahrnehmungsschwelle.**

6. Tiefgehende RL-Erkenntnisse und Systemanalysen

6.1 Das Subpixel-Wahrnehmungsproblem

Die mathematische Analyse der Bildauflösung liefert die Erklärung für das Verhalten der visuellen Agenten: * **Streckenbreite in Realität: 1.80 m** * **Bildbreite: 166 Pixel** $\implies$ **1 Pixel $\approx$ 10.8 mm** * **Simulationszeitschritt: $\Delta t = 0.0167$ s (60 Hz)**

Für einen sichtbaren Versatz von mindestens **1 Pixel** pro Zeitschritt ist eine Mindestgeschwindigkeit erforderlich:

$$v_{\min} = \frac{0.0108 \text{ m}}{0.0167 \text{ s}} \approx 0.64 \text{ m/s}$$

```
+-----+
|                                     |
|               DAS SUBPIXEL-DILEMMA IM ANFAHRBEREICH               |
|                                     |
| Geschwindigkeit  $v < 0.64$  m/s:      |
| Frame(t) == Frame(t-1) == Frame(t-2)  ==> Differenzenbild  $dI/dt = 0$  |
|                                     |
| Konsequenz:                      |
| - Der 3-Frame-Stack enthält drei identische Bilder.              |
| - Das CNN kann weder Geschwindigkeit noch Beschleunigung wahrnehmen. |
| - Der Agent verhält sich wie ein Blindflug-System und wird vom Reward bestraft. |
|                                     |
+-----+
```

6.2 Lösungsstrategien: Temporale Spreizung vs. Bildauflösung

Lösungsansatz	Konfiguration	Wahrnehmungsschwelle	Rechenzeit je Update	Replay-Buffer (500k)
Ausgangszustand	166 × 100, Stack $[t, t-1, t-2]$	0.64 m/s	55.4 ms	15.5 GB
Hebel 1: Temporale Spreizung	166 × 100, Stack $[t, t-4, t-8]$	0.16 m/s	55.4 ms ($\pm 0\%$)	15.5 GB ($\pm 0\%$)
Hebel 2: Höhere Auflösung	250 × 150, Stack $[t, t-1, t-2]$	0.42 m/s	104.6 ms (+89%)	34.9 GB (+125%)

Erkenntnis: Die temporale Spreizung (Frame Striding) ist der weitaus effizientere Hebel, da sie die Wahrnehmungsschwelle um den Faktor 4 senkt, ohne zusätzlichen Rechen- oder Speicheraufwand zu verursachen.

6.3 Replay-Buffer-Architektur und Speicheroptimierung

Standard-Puffer speichern gestapelte Bilder redundant ab (**3×** Bilddaten). Der in `src/utils/framestapel_buffer.py` implementierte Ringspeicher speichert pro Zeitschritt nur das Einzelbild und assembliert den Stack on-the-fly beim Batch-Sampling. Dies reduziert den Speicherbedarf um **Faktor 1.5** bei mathematisch bitidentischen Batches.

6.4 Korrekte Behandlung von Zeitschritt-Timeouts

Wird ein Timeout bei Frame 2000 fälschlicherweise als Terminalzustand markiert, setzt das Bellman-Update $V(s_{2000}) = 0$. Mit `handle_timeout_termination=True` wird der Wert korrekt gebootstrapped ($y_t = r_t + \gamma V(s_{t+1})$), was den künstlichen Werteverfall am Episodenende verhindert.

6.5 TensorBoard-Diagnostik: Critic-Divergenz vs. Ausreißer

- **Lauf 1432** : Der Critic-Loss explodierte nach Schritt 2.0M von **22.2** auf **26.670** (Faktor 1200) $\rightarrow$ echte Divergenz durch Q-Wert-Überschätzung.
- **Lauf 2210** : Der Peak von **875.7** Punkten spiegelte sich nicht im stochastischen Trainings-Return wider $\rightarrow$ statistischer Ausreißer einer deterministischen Einzelauswertung.

6.6 Messfallen und Rendering-Diskrepanzen

Ein Unterschied von nur **0.05% der Pixel** zwischen `render_mode="hidden"` (Training) und `render_mode="human"` (Visualisierung) kippte die Lenkentscheidung des CNNs von **+0.30** auf **-0.80**. Evaluationen müssen strikt dieselbe Bildpipeline wie das Training durchlaufen.

7. Übertragung in die 3D-Simulation (NVIDIA Isaac Sim)

7.1 Aufbau der 3D-Simulationsumgebung in Isaac Lab [Platzhalter]

Beschreibung der 3D-Szene, USD-Modelle der Carrera-Hybrid-Strecke und des Fahrzeugs, PhysX-Kollisionsparameter.

[!NOTE] Platzhalter für detaillierte Ausgestaltung durch 3D-Ergebnisse: Hier werden die USD-Szenenstrukturen, Kamera-Setups und PhysX-Reibwerte eingetragen.

7.2 Asymmetric Actor-Critic und Domain Randomization

In der 3D-Simulation wird die Teacher-Student-Architektur durch einen **Asymmetric Actor-Critic** erweitert: * **Critic**: Erhält während des Trainings Zugriff auf den vollständigen 3D-Ground-Truth-Zustand (exakte Position, Orientierungsquaternion, lineare und rotatorische Geschwindigkeiten, Reifenschlupf). * **Actor**: Erhält ausschließlich die synthetisierten 3-Kanal-Kamerabilder und Onboard-Telemetriedaten.

Zur Vorbereitung des Sim-to-Real-Transfers werden Reibwerte ($\mu \in [0.6, 1.2]$), Beleuchtungsstärken, Kamerarauschen und Latenzen domain-randomisiert.

7.3 Evaluierung und Sim-to-Real Ausblick [Platzhalter]

Vergleich der Rundenzeiten und Konvergenzgeschwindigkeiten zwischen 2D-Prototyp und 3D-Isaac-Sim-Training.

8. Fazit, Learnings und Ausblick

8.1 Zusammenfassung der Ergebnisse

Die vorliegende Arbeit demonstriert die erfolgreiche Konzeption und Validierung einer sample-effizienten Reinforcement-Learning-Pipeline für Miniatur-Rennfahrzeuge in einer 2D-Simulationsumgebung. Durch die Kombination aus privilegiertem Lehrer-Training (Raycasts), stochastischer Rausch-Injektion (Recovery Manöver) und visuellem Vortraining (Behavioral Cloning) konnte ein robuster Workflow geschaffen werden, der teure Trial-and-Error-Iterationen in 3D-Umgebungen überflüssig macht.

8.2 Zentrale Erkenntnisse (Key Learnings)

1. **Wahrnehmung vor Algorithmus:** Das Scheitern visueller Agenten lag nicht an SAC oder der Fahrzeugdynamik, sondern am Subpixel-Wahrnehmungsproblem bei niedrigen Geschwindigkeiten.
2. **Effizienz von Frame Striding:** Temporale Spreizung ($t, t - 4, t - 8$) löst die Wahrnehmungsschwelle ohne Rechenzeit-Overhead.
3. **Notwendigkeit von Noisy Demonstrations:** Reines Behavioral Cloning auf idealen Trajektorien führt unweigerlich zum Covariate Shift; nur verrauschte Demonstrationen vermitteln Abfangkompetenz.
4. **Speicher- und Render-Disziplin:** On-the-Fly Replay-Buffer und bitidentische Renderpipelines sind essenziell für reproduzierbares Deep RL.

8.3 Ausblick auf das Folgeprojekt

Aufbauend auf diesen 2D-Ergebnissen umfasst die Roadmap für das Folgeprojekt: * Vollständige Implementierung der Teacher-Student-Pipeline in **NVIDIA Isaac Lab**. * Training mit Asymmetric Actor-Critic unter intensiver Domain Randomization. * Sim-to-Real Transfer auf reale Carrera-Hybrid-Fahrzeuge mit Deckenkamera-Tracking.

Quellenverzeichnis

1. **Cai, P. et al. (2021):** *Vision-Based Autonomous Car Racing Using Deep Imitative Reinforcement Learning*. IEEE Robotics and Automation Letters (RA-L), Vol. 6, No. 4, S. 7262–7269.
2. **Chen, D. et al. (2019):** *Learning by Cheating*. In: Conference on Robot Learning (CoRL 2019), Proceedings of Machine Learning Research (PMLR), Vol. 100, S. 66–75.
3. **Wurman, P. R. et al. (2022):** *Outracing champion Gran Turismo drivers with deep reinforcement learning*. Nature, Vol. 602, S. 223–228.
4. **Amini, A. et al. (2020):** *Learning Robust Control Policies for End-to-End Autonomous Driving From Data-Driven Simulation*. IEEE Robotics and Automation Letters (RA-L), Vol. 5, No. 2, S. 1143–1150.
5. **Haarnoja, T. et al. (2018):** *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. In: International Conference on Machine Learning (ICML 2018), S. 1861–1870.
6. **Schulman, J. et al. (2017):** *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.
7. **Mnih, V. et al. (2015):** *Human-level control through deep reinforcement learning*. Nature, Vol. 518, S. 529–533.
8. **Ross, S. & Bagnell, J. A. (2011):** *A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning*. In: International Conference on Artificial Intelligence and Statistics (AISTATS 2011), S. 627–635.
9. **Fujimoto, S. et al. (2018):** *Addressing Function Approximation Error in Actor-Critic Methods*. In: International Conference on Machine Learning (ICML 2018), S. 1587–1596.

Abbildungs- und Tabellenverzeichnis

Tabellenverzeichnis

- **Tabelle 1:** Vergleich der physikalischen Modellierung (`force_drag_v1` vs. `measured_v2`) – *Kapitel 3.2*
- **Tabelle 2:** Quantitativer Vergleich aller Haupttrainingsläufe (PPO vs. SAC, Lidar vs. Vision) – *Kapitel 5.2*
- **Tabelle 3:** Vergleich der Lösungsansätze für das Subpixel-Wahrnehmungsproblem – *Kapitel 6.2*

Abbildungsverzeichnis

- **Abbildung 1:** Gesamtübersicht der Cross-Modal Privileged Distillation Pipeline – *Kapitel 1.1*
- **Abbildung 2:** Schematischer Ablauf des Privilegierten Lernens (*Learning by Cheating*) – *Kapitel 2.4*
- **Abbildung 3:** Die 4 Phasen der Teacher-Student Trainingspipeline – *Kapitel 4*
- **Abbildung 4:** Das Subpixel-Wahrnehmungsdilemma im Anfahrbereich – *Kapitel 6.1*

Anhang

Anhang A: Struktur und Workflow im Jupyter Notebook

Das Jupyter Notebook `Notebook_prototyping.ipynb` dient als interaktive Arbeitsumgebung: * **Zellen 0–15:** Streckenerfassung via OpenCV, interaktives HSV-Tuning, Mittelliniengenerierung, manuelles Fahren mit WASD. * **Zellen 16–26:** Initialisierung von `Carrera2DEnv`, Lidar-Visualisierung, Konfiguration der `VirtualCamera`. * **Zellen 27–49:** Baseline-Trainings (Raycasts, Single-Frame, Hybrid, Stacked Crop) und GFLOPS-Benchmarks. * **Zellen 50–63:** Teacher-Student Pipeline (Datensammlung mit Noise, CNN-Pretraining, SAC-Fineteuning). * **Zellen 64–70:** Diagnose- und Beobachtungszellen unter Original-Physik mit Telemetrie-Einblendung.

Anhang B: Wichtige Hyperparameter der Trainingsläufe

```
{
  "algorithm": "SAC",
  "policy": "CnnPolicy",
  "learning_rate": 5e-5,
  "buffer_size": 500000,
  "batch_size": 512,
  "gamma": 0.99,
  "tau": 0.005,
  "ent_coef": "auto",
  "train_freq": 1,
  "gradient_steps": 1,
  "handle_timeout_termination": true,
  "camera_resolution": [166, 100],
  "n_stack": 3,
  "max_steer_change": 0.5
}
```